

# Self-Organized Criticality: Sandpile Model on Undirected Graph, Influence of Topology and Dissipation Rules on Avalanches

## 1. Introduction

In everyday life, person's outburst is rarely caused by a single large event. Rather, the tension builds up from small frustrations over time. When certain threshold is reached, even a minor additional stress can trigger an outburst. That redistributes the tension to others in their social circle. This may in turn cause further reactions, leading to a cascade of reach and size which are difficult to predict. A local event may therefore propagate through a network, depending on individual thresholds and the shape of the network.

This may seem irregular or unpredictable, but many complex systems exhibit stable statistical properties when the number of observation is large. This phenomenon is explained by the law of large numbers [1] which applies, for example, to repeated coin tosses, dice rolls and also many natural phenomena such as average temperature.

Furthermore, if many independent effects contribute to an outcome, the resulting distribution is often well approximated by a normal distribution according to central limit theorem [1]. A common example is human height which clusters around an *average* with roughly symmetric variation.

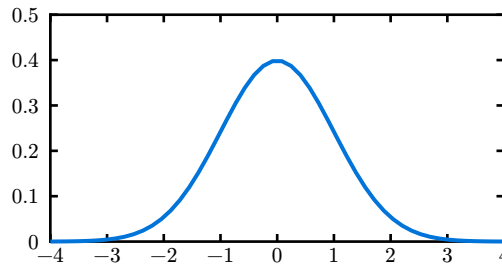


Figure 1: Normal distribution plot.

Contrary to this, there are systems which could not be described by normal distribution [2]. One example are the forest fire sizes [3]. If they were to adhere to normal distribution, there would be a known *average forest fire size*. Even though most forest fires are small, there is nothing limiting the potential size. This is one of the key properties of systems better described by the power law, scale-invariance.

Normal distribution is exponentially bounded, meaning that most of values are distributed within small range around the center (mean) and values far from it are very improbable. The probability density function of normal distribution with mean  $\mu \in \mathbb{R}$  and variance  $\sigma^2 > 0$  is

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/(2\sigma^2)}. \quad (1)$$

As opposed to this, the power law is *heavy tailed*. Its exponential character described in Equation (2) result in a higher probability of extreme values. That corresponds with the scale-invariance: Any (forest fire) size is realistically possible, even though smaller sizes are more likely. Power law exhibit linear relationship between  $\log f(x)$  and  $\log x$ . In a  $\log\text{-}\log$  plot it forms a straight line, as can be seen on Figure 2.

$$f(x) = ax^{-k} \quad (2)$$

$k \dots$  constant exponent

The emergence of the power law distribution is often connected to systems critical state. Consider phase transition of water from liquid to vapor. At the liquid-vapour boundary curve, both liquid and vapour can coexist but are distinctly separate. The boundary terminates at some critical temperature and critical pressure at which the critical point of a system is defined. While the system is in the critical state, the water is not in a single state.

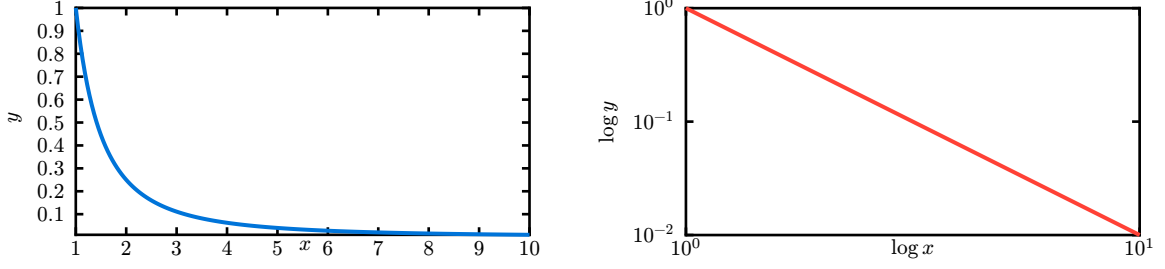


Figure 2: Function  $y = x^{-2}$  plotted on linear scale (left) and on log-log scale (right) on interval  $\langle 1, 10 \rangle$ .

Systems in critical state are most responsive to input and behave unpredictably [4]. This is the case for phase transition and forest fires. However, there is a significant difference. Criticality can be reached in many systems by fine-tuning certain parameters. In the case of phase transition the parameters are pressure and temperature. However, there exist systems which reach criticality independently of parameters.

A small local disturbance only manifest locally (e.g. throwing a pebble into a pond makes water rings from the origin, but the water on the other side of pond is not affected). However, in critical systems a small change can trigger response from afar (e.g. a forest fire might affect trees located hundreds of miles away).

The phenomenon of systems reaching criticality by themselves is called *self-organized criticality (SOC)*. The concept of SOC was first discovered in 1987 by Bak, Tang and Wiesenfeld and its properties were demonstrated on a sandpile model [5]. The original model was a cellular automaton on a square and cubic lattice.

In this thesis, we investigate the behaviour of the sandpile model on different graph structures. Specifically, on small-world networks which are a class of random networks. Before this generalization we introduce few necessary concepts from graph theory.

## 2. Graph Theory

An undirected graph is defined as a pair, containing set of vertices and set of edges. The graph  $G$  has  $n$  vertices and  $m$  edges,

$$\begin{aligned} G &= (V, E) \\ V &= \{v_1, v_2, \dots, v_n\} \\ E &\subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}, \quad |E| = m. \end{aligned} \quad (3)$$

The neighbourhood  $\mathcal{N}$  of a vertex  $v$  is a set of vertices connected to it via an edge,

$$\mathcal{N}(v) = \{u \mid \{u, v\} \in E\}. \quad (4)$$

The vertex degree  $d(v)$  is defined as the size of vertex neighbourhood,

$$d(v) = |\mathcal{N}(v)|. \quad (5)$$

Path between two vertices  $u$  and  $v$  is a sequence of vertices beginning in the first and ending in the second where every two vertices in path must be directly connected,

$$\begin{aligned} P_{u,v} &= (v_1, v_2, \dots, v_k), \quad v_1 = u, \quad v_k = v, \\ \{v_l, v_{l+1}\} &\in E \quad \forall l \in \{1, \dots, k-1\}, \end{aligned} \quad (6)$$

if there exist no such path, let  $P_{u,v} = ()$ .

Similarly, the path length is the number of *steps* (edges) needed to walk the path,

$$\text{len}(P_{u,v}) = \begin{cases} |P_{u,v}| - 1 & |P_{u,v}| \geq 2, \\ 0 & \text{else,} \end{cases} \quad (7)$$

if a path doesn't exist we define its length as zero.

The shortest path between vertices  $u$  and  $v$  is the path with minimal length among all paths connecting these vertices. Let  $\mathbb{P}(u, v)$  be the set of all paths between  $u$  and  $v$ ,

$$P_{u,v}^* = \min_{P \in \mathbb{P}(u,v)} \text{len}(P). \quad (8)$$

A graph has an average (shortest) path length,

$$\text{len}(G) = \frac{1}{n(n-1)} \sum_{u \neq v} \text{len}(P_{u,v}^*). \quad (9)$$

We say that two vertices  $u, v$  are connected if at least one path between them exist, i.e.

$$\text{len}(P_{u,v}) > 0. \quad (10)$$

The graph is connected if all its vertices are connected,

$$\forall u, v \in V, u \neq v : \text{len}(P_{u,v}) > 0. \quad (11)$$

A subgraph  $G'$  of graph  $G$  contains a subset of its vertices and edges,

$$G'(V', E') \subseteq G(V, E) \Leftrightarrow V' \subseteq V \wedge E' \subseteq E. \quad (12)$$

The graph component is a connected subgraph which is not part of any larger connected subgraph.

Complete graph  $K_n$  is a graph with  $n$  vertices and  $\frac{n(n-1)}{2}$  edges. This means that every two vertices are directly connected and the average path is one.

The measurement of how much do individual vertices in graph cluster together is called global clustering coefficient and it is based on triplets of nodes. One triplet is defined as three vertices with two or three edges between them. If the number of mutual edges is two, the triplet is called open, if it is three, the triplet is called closed. The global clustering coefficient of graph  $C(G)$  is defined as

$$C(G) = \frac{\# \text{ closed triplets}}{\# \text{ all triplets}}. \quad (13)$$

Small world network is a graph with certain characteristic [6]. It has high clustering coefficient and low distances. One example of small world network might be a human relationships network. The low distances might be expressed with respect to number of vertices as

$$\text{len}(G) \propto \log(|V|). \quad (14)$$

Small world networks tend to contain complete subgraphs (cliques). This is direct result of high clustering coefficient. Additionally, because the average path is small, they often contain *hubs* (vertices with high degree) which serves as connections between other vertices.

One of methods to quantify the *small-worldness* of a network is a small-world measure ( $\omega$ ) defined as follows [7]

$$\omega(G) = \frac{\text{len}(R)}{\text{len}(G)} - \frac{C(G)}{C(L)}. \quad (15)$$

Where  $G$  is the analyzed network,  $R$  is an equivalent random network (i.e. has the same number of vertices and degree distribution [7]) to  $G$ , and  $L$  is an equivalent (ring) lattice network to  $G$ . This method aims to quantify the formulating properties of small-world network: high clustering coefficient which is *ideal* in lattice networks, and low average path which is *ideal* in random networks.

### 3. Sandpile Model

The very original sandpile model is a cellular automaton on a lattice. Every cell can hold up to  $K$  grains, the height of a cell is a function of position  $z(x, y, \dots)$ . Fixed boundary conditions are used, on a boundary  $z = 0$  cannot be changed. In two dimensions  $z$  is updated as follows:

$$\begin{aligned} z(x, y) &\rightarrow z(x, y) - 4 \\ z(x \pm 1, y) &\rightarrow z(x \pm 1, y) + 1 \\ z(x, y \pm 1) &\rightarrow z(x, y \pm 1) + 1 \end{aligned} \quad (16)$$

if  $z$  exceeds (or meets) critical value  $K = 4$ . The system is initialized randomly but with  $z \gg K$ . That initial state or rather the size of a lattice along with  $Z$  (the height of all cells) is called *configuration*.

For simplicity we can imagine a simple chessboard and falling sand grains. Each grain falls on a chess square determined by a random distribution<sup>1</sup>. If the number of grains on any square reaches four, the sandpile topples and the four grains fall on adjacent squares, possibly causing additional squares to topple. In case the square does not have four adjacent squares (i.e. it is on the edge of chessboard) the remaining grains fall off the board and are therefore removed from the model.

We say that cell (square) is stable if it contains fewer grains than  $K$ . The system is stable if all of its cells are stable. If a grain is added to any stable cell, it can destabilize it resulting in an avalanche. Avalanche is the set of topplings which occur before the system is stabilized again.

The sandpile model is a dissipative dynamical system. The dissipation is demonstrating on the boundaries. When a grain leaves the chessboard (or enters a boundary cell) it dissipated from the system. Dissipation is an important property of the model. Without dissipation the model supersaturates, leading to one infinite avalanche.

We can express the square lattice (chessboard) as a graph  $G = (V, E)$  with the size given by<sup>2</sup>  $\sqrt{n}$ . For a boundary condition, the boundary vertices height is fixed to zero  $z = 0$ . Alternatively, we may imagine only one vertex, called *sink vertex*, with such condition. This sink is connected to all boundary vertices, thus creating the same effect. Please note, that this simplification creates a multigraph, because corner vertices and sink are connected via two edges. For the rest of this thesis we will consider the simplified version with one sink vertex and therefore a multigraph. However, it is trivial to create multiple sink vertices to restore a simple undirected graph.

---

<sup>1</sup>In the original model, all sand grains were distributed initially. It might be more obvious if we let grains fall one at a time for demonstration purposes.

<sup>2</sup>In a square lattice, the size<sup>2</sup>  $= n$ , which is the number of vertices in graph.



Figure 3: Comparison of square lattice sink vertices (blue). On the left there is a graph with sinks at the boundary - as in the original model. On the right there is a multigraph, the square lattice with a single sink connected to all vertices at the boundary which is hinted by outgoing edges.

All properties of the cellular automaton system holds on a square lattice. This model was later generalized from square lattice to arbitrary graph [8]. That generalization is important because it allows us to study the model's behaviour for different graph topologies.

The classical model has the following important property. The final configuration is independent of the order in which avalanches happen. This also implies that when adding two grains on different cells, the final configuration does not depend on the order of adding grains. It is because the addition of a sand grain can be seen as an operator, and these operators were proved to form an Abelian group [9].

This property was later shown to hold on finite (un)directed graphs [8], allowing us to study the behaviour of the model on different graph topologies, rather than studying the consequences of changing a toppling order.

### 3.1. Graph Topologies

Standard choice for the sandpile model graph is the square lattice with arbitrary size. This however does not realistically represent the model from the motivation story in Section 1 and other possible applications. That is why we introduce different graph topologies.

Before leaving square lattice entirely, we discuss toppling and dissipation rules on it. The original model presents a toppling rule described in Equation (16) and  $K = 4$ , where  $K$  is the number of grains which cause a single cell (vertex) to topple.

Dissipation rules on a general graph are more complex and further discussed later in Section 3.2. While the different definitions of  $K$  and toppling rules might be more deeply explored, we will only consider  $K$  being a function of a vertex,

$$K(v) = d(v), \quad v \in V. \quad (17)$$

The toppling rule is also expressed as a function of a vertex. Similarly to the original definition as cellular automaton, the height of a vertex is expressed as a function

$$\begin{aligned} z(v) &\rightarrow z(v) - K(v) \\ z(n) &\rightarrow z(n) + 1, \quad n \in \mathcal{N}(v). \end{aligned} \quad (18)$$

This simple rule states, that during the toppling process the original vertex loses as many grains as many neighbours it has and all its neighbours gain one. The benefit of this  $K$  and toppling rule is their simplicity.

For example, it might be interesting to set  $K = \text{const}$ , but that would create problems. If a vertex has fewer neighbours, it is unclear where the additional grains go during a topple. If it has more neighbours, does it artificially generate more grains, or distribute fewer grains between randomly selected neighbours?

Note that in these reasonings, the neighbourhood of a vertex  $\mathcal{N}(v)$  contains sink vertex, and possibly multiple times. The toppling rule will not work if  $K(v) = 0$ . We acknowledge this constraint but leave its resolution to dissipation rules.

With  $K$ , toppling rule and dissipation rules specified, we now continue onto the properties of different graph topologies. Square lattice is a regular graph, meaning the degree of all vertices is the same [10], if we consider sink vertex.

Regularity is not often found in human social networks, meaning graphs where vertices represent individuals and edges their mutual acquaintance. Purely random graph also would not be optimal since it is not regular. However, human networks are not completely random. Fortunately, a small-world network (better described in Section 2) are almost perfect.

The three most known random graph generators are the Erdős–Rényi (ER) model [11], Barabási-Albert (BA) [12] model and Watts-Strogatz (WS) model [6]. Each of these methods generates the graph differently resulting in various *small-worldness*.

The **ER model** was the first discovered. There are two variants, both of which start with graph with  $n$  vertices. The  $G(n, m)$  model chooses a graph uniformly from the set of all possible graphs with  $n$  vertices and  $m$  edges. The  $G(n, p)$  model creates all possible edges but only with the probability  $p$ .

The  $G(n, m)$  model ensures that any edge is equally likely and the number of edges is known before. This, however, does not provide the resulting graph being connected. The  $G(n, p)$  model ensures that every edge has a fixed probability of being present, also not ensuring connected graph.

Another drawback of this model is the low clustering coefficient. That is a consequence of all edges having the same probability, which is not usually seen in social networks and in turn in small-world networks.

The latest to be discovered is the **BA model**. The important observation BA made is that many observed networks are *scale-free*, meaning they have a power-law degree distribution. Their method of generating has two key concepts: growth and preferential attachment.

Growth simply means that the graph is build up from small number of vertices. Preferential attachment means for a vertex, the higher degree it has, the more likely it is to be connected to new vertex.

The algorithm for generating the graph is, simply put, a loop in which one vertex is added to the graph and connected to  $m$  vertices. They are chosen from all vertices with probability

$$p(v) = \frac{d(v)}{\sum_u d(u)}, \quad u, v \in V, \quad (19)$$

where  $v$  is the examined vertex and  $u$  represents all vertices. The newly added vertex is not yet considered. The sum of all  $p$  is equal to one and edges are selected by repeatedly ( $m$  times) generating random number from 0 to 1 and taking corresponding edge (the original paper does not specify case when one edge is selected multiple times).

The algorithm takes  $G_0, n, m$  where  $m$  is the number of edges for each new vertex and  $n$  represents the number of vertices in final graph. The first argument is a starting graph with at least  $m$  vertices. In practice,  $G_0$  is usually chosen as a complete graph  $K_m$  of size  $m$ .

This algorithm yields a quick emergence of hubs, vertices with high degree. BA provides example i.e., newly create webpage is more likely to link to well-known websites which already have high degree.

As the consequence of hubs' existence, the average path is short. However, interesting property of BA model is the dependance of clustering  $C$  on  $N = |V|$ ,

$$C \sim \frac{\ln(N^2)}{N}. \quad (20)$$

This behaviour is different from small-world network where clustering is independent of size [12].

This model generates a connected graph, because when new vertex is added, it is connected to exactly  $m$  existing vertices.

In the **WS model**, authors have also defined the small-world measure ( $\omega$ ) discussed in Equation (15). WS argued that ER model lacks two important properties often observed in small-world networks: high clustering coefficient and the convergence of degree distribution to power law. Their model was designed to address the first issue.

The generation algorithm is simple: 1. start with a regular ring lattice, 2. rewire (i.e. remove the edge but add another from the same vertex) every edge with probability  $\beta$ . The idea is to preserve the short average path present in ER model, but simultaneously create high clustering.

This is done by interpolating between the regular ring lattice (which has high clustering coefficient) and ER model. When  $\beta = 0$ , no edge is rewired and the ring lattice holds. It cannot approach the ER model, because rewiring does not remove edges and therefore the graph is connected - not a guaranteed property in ER model.

The algorithm for WS graph generation needs these parameters: number of vertices  $N$ , mean degree  $K$  (even integer) and rewiring probability  $\beta$ . When  $\beta = 1$  it approaches a structure close to ER  $G(n, p)$  model with  $p = \frac{K}{N-1}$ .

One necessary constraint on parameters is that  $N > K$ . It is widely agreed upon to use  $N \gg K$  (e.g.  $N = 10K$ ) when constructing graph.

While this method was designed to have high clustering coefficient and low average path length, the degree distribution is not power law as seen in BA model. This divergence from observable small-world networks (e.g. airport networks) is reflected in all vertices having similar degree.

Either of these methods of generating small-world network has its imperfections. In the interactive tool developed for the needs of this thesis are implemented all mentioned methods. Note, that from these only WS method focus solely on small-worldness, BA method is focused on scale-free networks and ER method on random graphs.

### 3.2. Dissipation Rules

With the introduction of arbitrary graph we must reconsider dissipation rules. The square lattice has explicit boundaries which serve as sinks. However, many graphs do not have an implicit boundary. The existence of sink is fundamental for the model not to supersaturate.

A dissipation rule determines how many times a vertex should be connected to the sink. There is not a single rule which would be generally applied to any graph. We discuss different rules, their properties and analogies. Dissipation rules are not well-known thus their names are created for the purposes of this thesis.

We can present a dissipation rule as a function of vertex returning an integer which denotes how many times a vertex should be connected to the sink.

$$D : v \rightarrow \mathbb{N} \quad . \quad (21)$$

For example, the graph representation of the original model uses a **Fill to Four** rule. It states that the degree of any vertex must be four or more. The sink vertex is connected to all vertices as many times

it is needed to fulfil that rule. For the square lattice, only vertices on the edge are connected (exactly once except for corner vertices, which are connected twice).

$$D(v) = \begin{cases} 4 - d(v) & d(v) \leq 4, \\ 0 & . \end{cases} \quad (22)$$

For different graphs though, this rule might be insufficient. It does not ensure dissipation. This rule does not consider the vertex degree if it is higher than four.

To ensure dissipation on any graph, the rule must connect all vertices to the sink at least once. That is because there might exist vertices which are not connected to any other vertex. A problem arises with such vertex if it were not connected:  $d(v) = 0 = K$  meaning it should trigger an avalanche but that itself is nonsensical. Dissipation rules must prevent this from happening.

If we consider connected graphs the dissipation condition softens. It is guaranteed that for all pairs of vertices exist at least one path therefore only one sink is sufficient to ensure dissipation. This might be useful for the sandpile model is only interested on connected graphs where they exhibit critical behaviour.

One possible rule which guarantees dissipation on any graph is **All Once**. It ensures just the necessary condition that every vertex must be connected to sink at least once but nothing more. The rule is fair to all vertices as the connection to the sink is not exclusive to vertices of some characteristic<sup>3</sup>.

$$D(v) = 1 \quad . \quad (23)$$

One rule sufficient for connected graph is **As Many As Neighbours**. This rule connects every vertex to sink proportionally to its neighbourhood size. When considering connected graph this rule ensures dissipation and even provide a connection to the sink to all vertices.

$$D(v) = d(v) \quad . \quad (24)$$

The *Fill to four* rule may be seen as a particular case of the general **Fill to N** rule. The specific case where  $N = 4$  is useful for (two dimensional) square lattice, but for three dimensional lattice, for example, is more adept  $N = 6$ . The generalized rule with static value (e.g. 4 or 6) might be central for particular graph topologies (e.g. the lattice).

However, if we permit  $N$  to vary based on specific graph parameters, we introduce **Fill up** rule, where  $N$  is equal to the degree of vertex with largest neighbourhood.

$$N = \max_{v \in V} d(v) \quad (25)$$

Static  $N$  does not ensure dissipation on general graph, because it might only contain vertices with degree higher or equal  $N$ . Contrary to that, dynamic  $D$  ensures dissipation on any graph with either single vertex with the highest degree (higher than 0) or multiple highest degree vertices which are not the only vertices of one graph component, the rule being:

$$D(v) = \begin{cases} N - d(v) & d(v) \leq N , \\ 0. & \end{cases} \quad (26)$$

Every rule represents a different approach which might not be applicable to all graph topologies. Below are listed necessary requirements on graph for all dissipation rules in order to ensure dissipation:

**Fill to N** Each graph component has at least one vertex with degree  $d(v) < N$ .

---

<sup>3</sup>Possible characteristics include vertex neighbourhood size, shortest path by which it reaches all other vertices and more.



**All once** No requirement.

**As many as neighbours** There is no vertex with degree zero, i.e. the graph is connected.

**Fill up** Single highest degree vertex or multiple highest degree vertices which are not the only vertices of one graph component.

Different graph topologies are suitable for various subset of dissipation rules, as shown in Table 1. Note that specific graph generated by any random method may be suitable for additional dissipation rules, the table covers the general case.

|                              | Square lattice | ER        |            | BA             | WS               |
|------------------------------|----------------|-----------|------------|----------------|------------------|
|                              | $G(n)$         | $G(n, p)$ | $G(n, m)$  | $G(G_0, n, m)$ | $G(N, K, \beta)$ |
| <b>Fill to N</b>             | $N \geq 3$     | ✗         | ✗          | ✗              | ✗                |
| <b>All once</b>              | ✓              | ✓         | ✓          | ✓              | ✓                |
| <b>As many as neighbours</b> | ✓              | ✗         | ✗          | $m \geq 1$     | ✓                |
| <b>Fill up</b>               | ✓              | $p = 1$   | $m \geq 2$ | ✓              | ✓                |

Table 1: The suitability comparison for different dissipation rules on graph topologies. Rule is suitable (✓), not suitable (✗) or only suitable for graphs with extra constraints. Standard constraints discussed in Section 3.1 are put on BA and WS models.

The square lattice is suitable for all discussed dissipation rules. Each vertex has exactly four neighbours, except for border vertices which only have three and corner vertices with only two. For *Fill to N*, the smallest possible  $N$  must be higher than the smallest degree in graph. Thus the  $N \geq 3$  condition.

It might seem as *Fill to N* rule should be suitable for more graph topologies, but the parameter  $N$  is set to be static and therefore cannot be set based on graph properties. For the remaining topologies, the only plausible way would be to set  $N$  equal the number of vertices or edges, whichever is lesser.

The *All once* rule is designed to work with any graph, which could be confirmed in the table. The fact it does not have any requirement makes it a rather useful dissipation rule.

Both ER methods does not guarantee connected graph. Therefore the *As many as neighbours* rule requirement is not fulfilled. On the other hand, the *Fill up* rule might be used with some conditions. In the  $G(n, p)$  method, to ensure at least one edge existing, the probability of edge creation must be set to 100% (creating a complete graph  $K_n$ ). For  $G(n, m)$  the condition is weaker, as the argument  $m$  is the number of edges in final graph.

Both BA and WS methods create a connected graph, passing the requirement for *As many as neighbours*. The condition for BA is rather symbolic: if  $m = 1$ , the graph is its own spanning tree. Any  $m$  smaller than that does not result in connected graph.

## 4. Observed Behaviour

In the original paper, BTW [5] focused, among other things, on cluster sizes. They define clusters as cells (vertices) that can be reached through the domino process originated by the toppling of only a single cell. They found that the distribution of cluster sizes obeys a power law.

The goal of this thesis is to explore the behaviour of the sandpile model on discussed graphs and with different dissipation rules. The main focus is the distribution of avalanche sizes (equivalent to cluster sizes).

BTW examined many stable configurations repeatedly and recorded all clusters, therefrom obtaining cluster sizes. Avalanche sizes, on the contrary, are only sizes of those clusters, which were toppled. However, the size of an avalanche might be bigger than the cluster size.

As shown in Figure 4, during an avalanche, additional vertices might become unstable and be toppled. Furthermore, one vertex can be toppled multiple times during a single avalanche. However, for a large enough graph and a number of grains, cluster and avalanche sizes all have a power-law distribution.

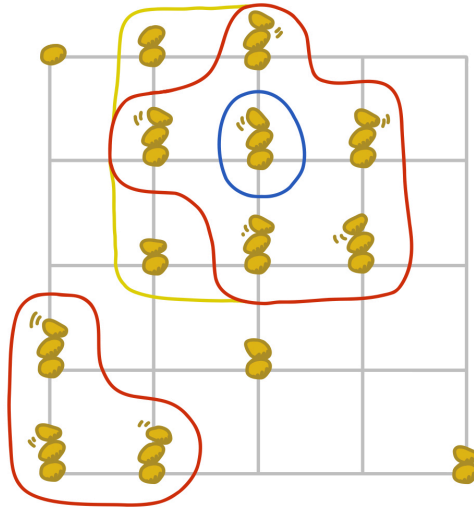


Figure 4: Example clusters on a square lattice of size  $5 \times 5$ . Every cluster is bounded by a red border. There are two clusters, one of size 3, the other of 6. Avalanches are represented as yellow borders extending the red. Avalanche sizes are 3 and 8 respectively. If a grain is dropped onto the vertex with blue outline, the avalanche size is 9.

To verify, a square lattice with  $n = 100 \times 100$  was created, and over one million grains were dropped. Figure 5 shows the measured data alongside a power-law fit. Over multiple runs, the coefficient in the power law is approximately  $-1.2$  which is comparable to  $-\frac{5}{4}$ , the numerically found value for square lattice [13]. As can be seen, larger avalanches happened slightly less often than predicted, skewing the fit.

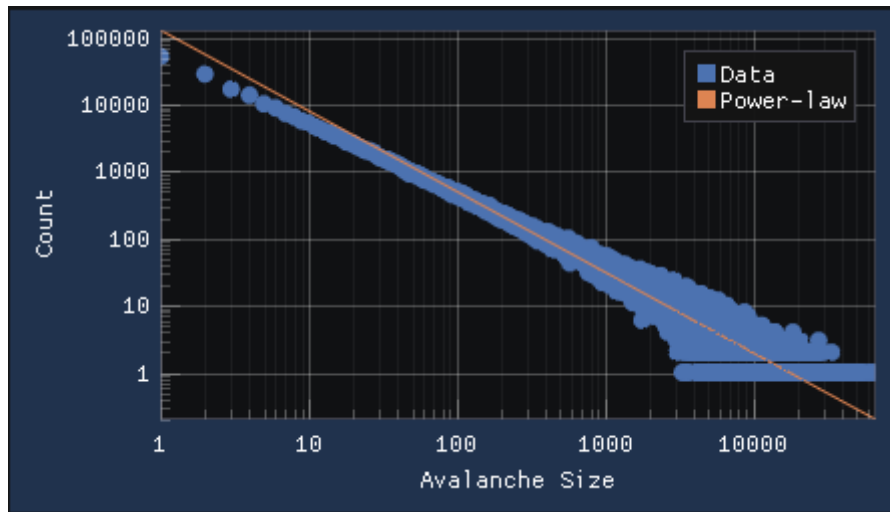


Figure 5: The distribution of avalanche sizes on a square lattice shown on a log-log plot.

The sandpile model does behave as expected on a standard square lattice graph with classic *Fill to Four* rule. The question is, does a power-law hold for different graph topologies and dissipation rules?

The ER model is the least to be expected to have similar properties to square lattice, as it does not guarantee a connected graph. For  $G(n, p)$ ,  $n = 100$ ,  $p = 0.5$  the resulting graphs are shown in Figure 6, in both log-log and linear plot.

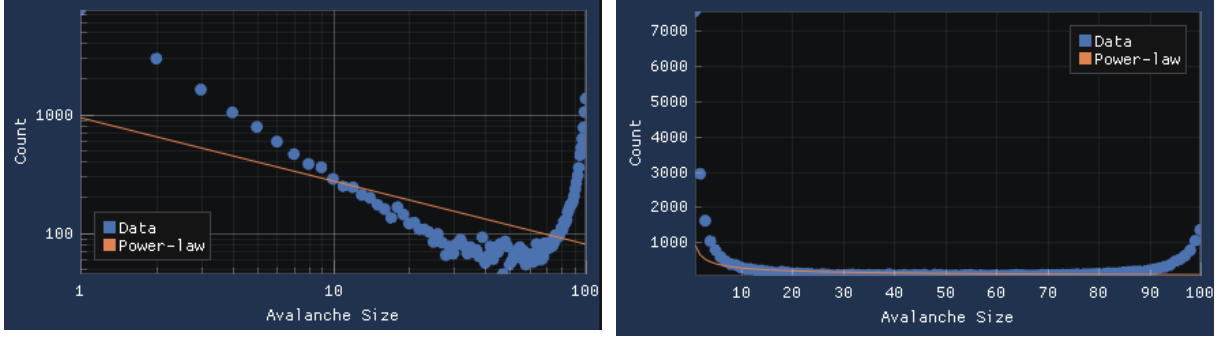


Figure 6: The distribution of avalanche sizes on a ER  $G(n, p)$  graph. The log-log plot on the left, linear plot on the right.

Surprising at first is the size of the greatest avalanche, which is equal to  $n$ . However, even though this model does not guarantee connected graph, the probability of not being so is very small. In fact, for mentioned parameters, the probability of one fixed vertex being disconnected is

$$p = \left(\frac{1}{2}\right)^{100}. \quad (27)$$

The distribution doesn't seem to fit a power-law on the whole interval, but rather it resembles a distribution more fitting a complete graph. However, if the edge existence probability is set to be lower, the distribution changes a lot.

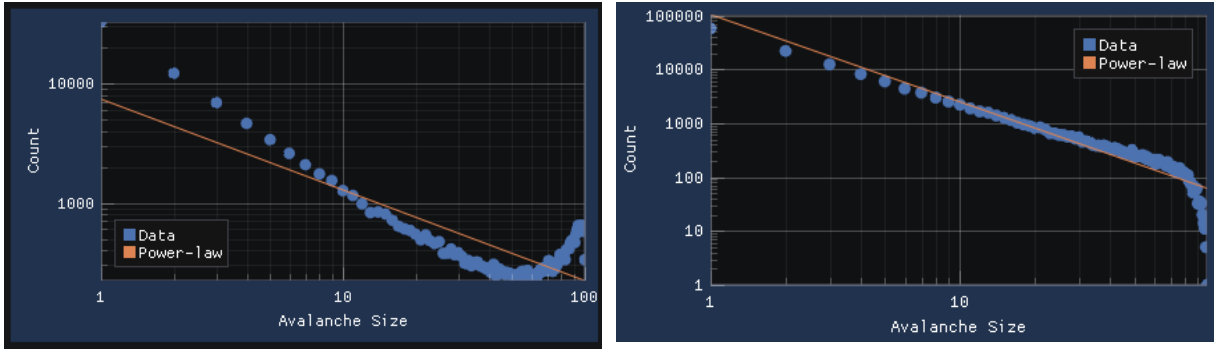


Figure 7: ER  $G(n, p)$  distribution with (from left)  $p = 0.25$  and  $p = 0.1$ .

That shows, the ER  $G(n, p)$  model can be fine-tuned to reach power-law in avalanche size distribution. This is, however, not self-organized then. Or rather, the ER graph does not ensure the emergence of self-organized criticality of the sandpile model with arbitrarily chosen parameter  $p$ . The ER  $G(n, m)$  model exhibits equivalent behaviour with the parameter  $m$ .

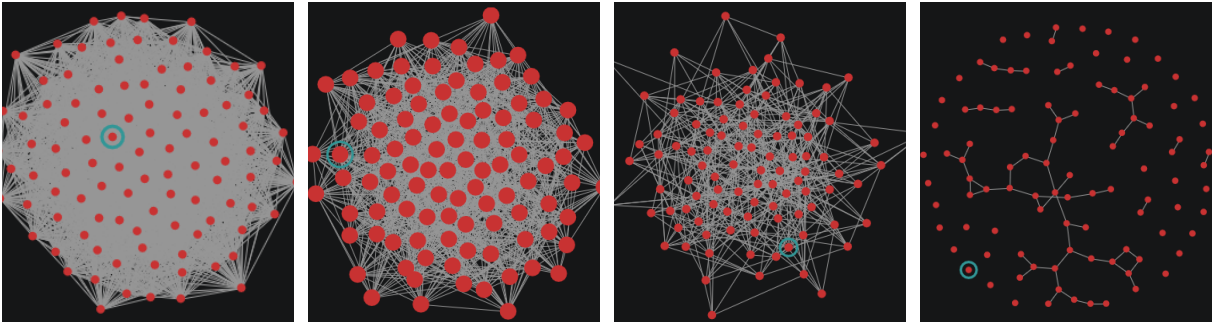


Figure 8: Comparing ER  $G(n, p)$  graphs with (from left)  $p = 0.5$ ,  $p = 0.25$ ,  $p = 0.1$  and  $p = 0.01$ .

The BA model is interesting for its avalanche origins distribution. That is another valuable metric, as the grains are distributed uniformly amongst all vertices, some of those are more likely to start an

avalanche. Note that even an avalanche of size one (i.e. no chain reaction, only one vertex topples) is included in this statistic.

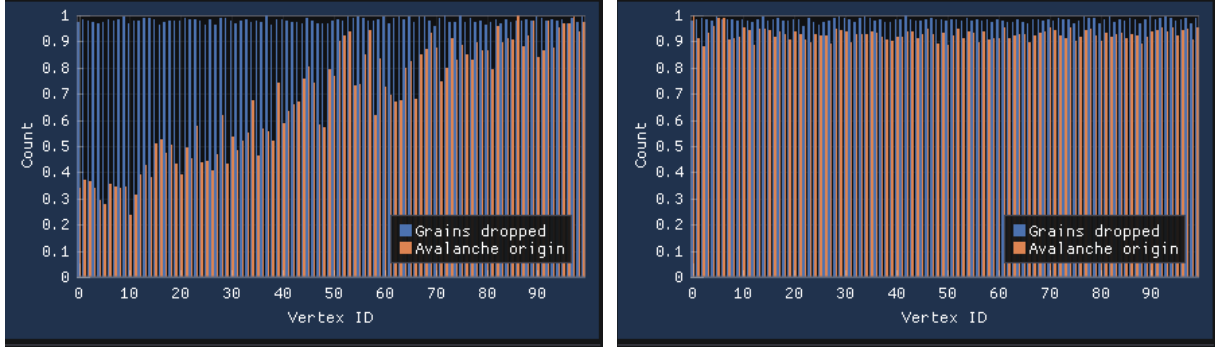


Figure 9: The avalanche distribution graph for BA generated graph with  $n = 100, m = 10$ . Blue bars show normalized count of grains dropped to vertex. Orange bars show normalized count of avalanches started at that vertex. Vertices with lower ID were added to the graph earlier during generation.

All once dissipation rule on the left, fill up rule on the right.

The plot at Figure 9 demonstrate the preferential attachment of the BA model. It is apparent that vertices which were added to the graph earlier started far fewer avalanches. That comes from the fact that these vertices have higher degree on average. It is more likely some vertex with lower degree starts an avalanche, albeit smaller in size.

That was recorded with the dissipation rule *All Once*. The change when this rule is swapped for *Fill Up* is peculiar. With that, all vertices have the same degree and the uniform distribution of grains is reflected on the distribution of avalanche origins.

While trying to fit a power-law for the avalanche size distribution does not make sense for general ER, it might for BA. If the *All Once* rule is applied (which may be seen on Figure 10), the distribution becomes too unpredictable with higher  $m$  and avalanche size. Therefore, letting  $m$  be small (preventing the graph to become too connected) is better.

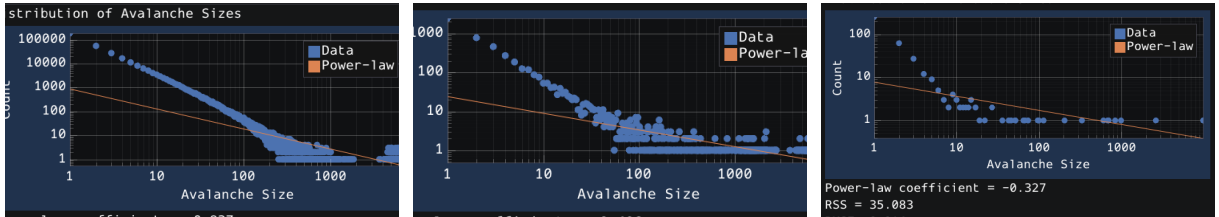


Figure 10: Comparing BA method with *All Once* dissipation rule where the parameters in  $G(G_0, n, m)$  are:  $n = 10.000, G_0 = K_m$  and  $m$  is 10, 100 and 1000 from the left respectively.

The rule *Fill Up* becomes more interesting. While it does make the avalanche origins distribution uniform, it also makes the observed data more consistent with power-law fit even for higher  $m$ .

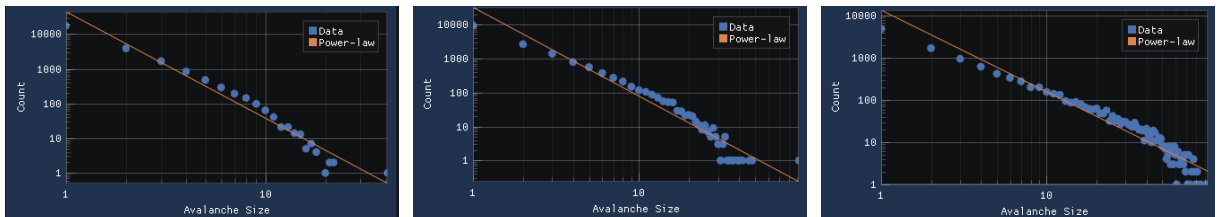


Figure 11: Comparing BA method with *Fill Up* dissipation rule where the parameters in  $G(G_0, n, m)$  are:  $n = 100, G_0 = K_m$  and  $m$  is 10, 25 and 50 from the left respectively.

The third rule, *As Many As Neighbours*, combines the properties of previous two rules. The avalanche origin distribution is similar to *All Once* rule, while the avalanche size distribution resembles *Fill Up* method more.

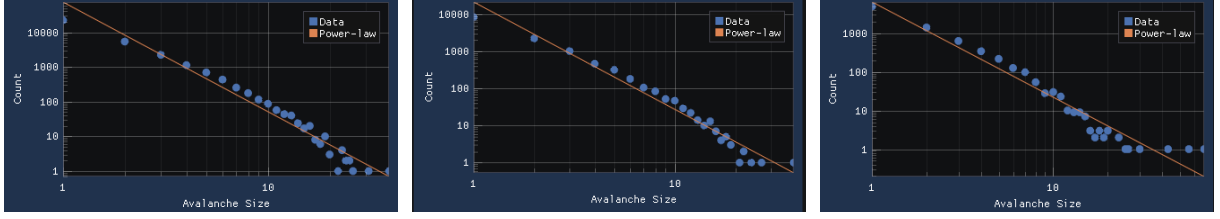


Figure 12: Comparing BA method with *As Many As Neighbours* dissipation rule where the parameters in  $G(G_0, n, m)$  are:  $n = 100$ ,  $G_0 = K_m$  and  $m$  is 10, 25 and 50 from the left respectively.

The degree of all vertices is higher,

$$d_{\text{As Many As Neighbours}}(v) = 2(d_{\text{All Once}}(v) - 1) , \quad (28)$$

because the sink is connected more times to each vertex. It does, however, preserve the difference between vertices, thus the similarity of avalanche origin distribution with *All Once* rule. Doubling the vertex degree, effectively, the vertex degrees become closer. That is the reason for avalanche size being similar to *Fill Up* (where all vertex degrees are exactly the same).

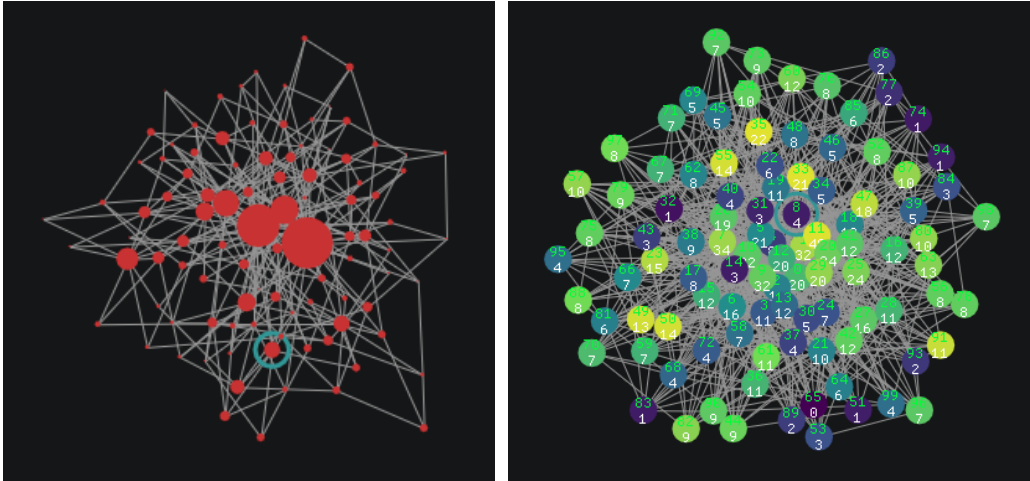


Figure 13: The difference between BA models with parameter  $n = 100$  and variable  $m$ . On the left,  $m = 3$ . On the right,  $m = 10$ . The right image shows alternative approach to showing count of grains on vertices. The color gradient, from purple to yellow to green, represents relative number of grains instead of size. The green number is vertex ID, the white is grains count.

We can use BA method to observe the behaviour of the sandpile model on a spanning tree. That could be generated when  $n$  is set to one. Spanning trees generated via this method still have preferential attachment.

As can be seen on Figure 14, the power-law distribution in avalanche sizes is present. The reason might be the similar degree of all vertices as most of them are only connected to one or two others.

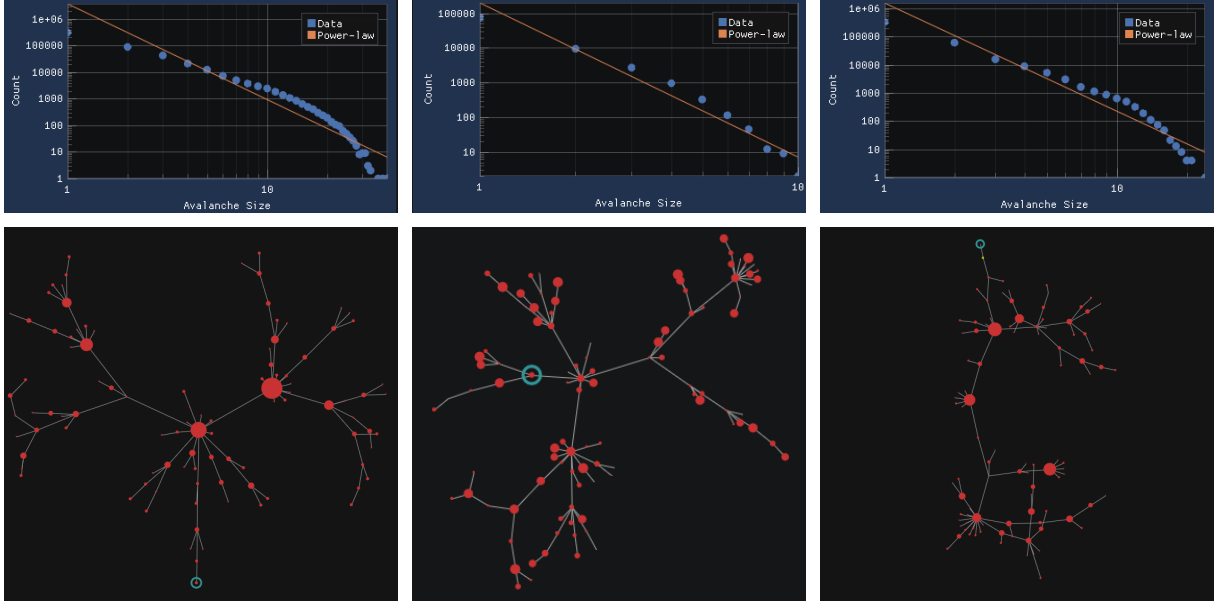


Figure 14: Comparing spanning trees (created by BA method with  $m = 1$ ) avalanche size distributions with regard to dissipation rule. From the left, *All Once*, *Fill Up*, *As Many As Neighbours*.

The WS model is the nearest to a small world network from all discussed models. The distribution of avalanche sizes is almost identical for any chosen  $\beta$  and dissipation rule. For the creation algorithm,  $G(N, K, \beta)$ ,  $K$  must be much smaller than  $N$  otherwise the graph is closer to a connected graph than to small world network.

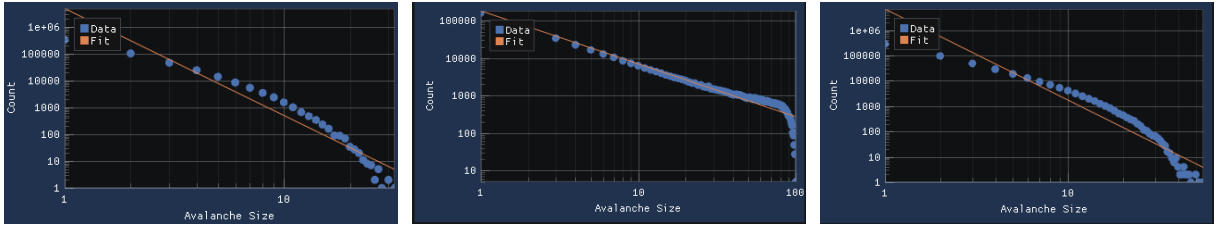


Figure 15: Avalanche size distribution comparison for WS method with parameters  $G(N = 100, K = 2, \beta)$  where  $\beta$  is 0.25, 0.5 and 0.9 from the left. The dissipation rules are *As Many As Neighbours*, *All Once* and *Fill Up* from the left respectively.

To measure the impact of different dissipation rules and graph topologies, multiple simulations were run and the average value of avalanche size distribution power-law fit entered into Table 2. Note that the power-law fit is computed in the interactive tool as a power-law regression. It assumes data follows

$$y = ax^k, \quad (29)$$

and takes the logarithm of both sides

$$\log(y) = k \log(x) + \log(a), \quad (30)$$

thus transforming it to a linear regression in log-log space. This kind of regression is prone to outliers. When measuring, sometimes the avalanche size distribution does exhibit a power-law only on a sub-range. One example is shown on Figure 16. The fit is not very accurate there since it includes all sizes. As a measure for the fit, the residual sum of squares (RSS) and root mean square error (RMSE) were used.

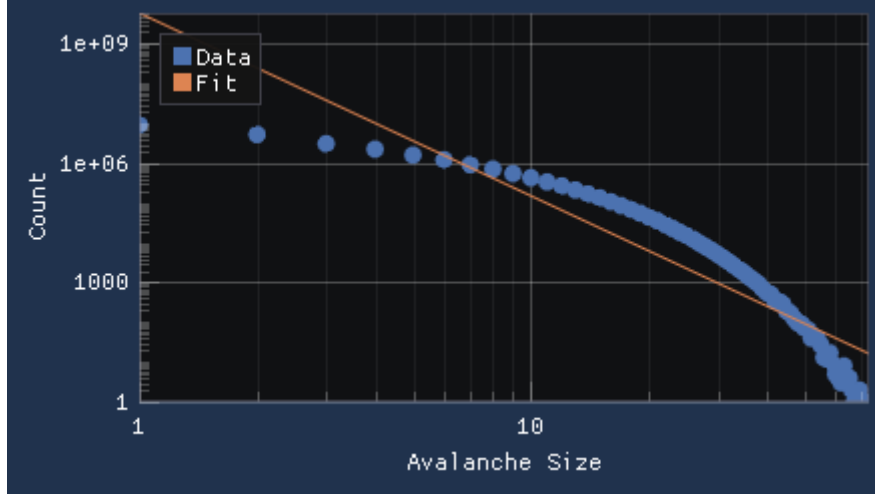


Figure 16: Avalanche size distribution of square lattice  $G(n = 1000)$  with *All Once* rule after  $\approx 10^9$  dropped grains.

One interesting observation is that for square lattice, where the coefficient is known (discussed at the beginning of this section) to be  $\approx \frac{5}{4}$ , the measured coefficient is higher even for the *Fill Up* rule (which is defacto *Fill to Four* on square lattice).

The reason for it is the issue with the fit discussed previously. All combinations of graphs and dissipation rules exhibit that to a certain degree.

|                                                 | Dissipation rules  |                              |                    |
|-------------------------------------------------|--------------------|------------------------------|--------------------|
|                                                 | <i>All Once</i>    | <i>As Many As Neighbours</i> | <i>Fill Up</i>     |
| <b>Square lattice</b>                           | $-3.955 \pm 1.463$ | $-4.651 \pm 1.487$           | $-3.060 \pm 1.48$  |
| <b>ER <math>G(n, p = 0.01)</math></b>           | $-2.532 \pm 0.719$ | $-3.961 \pm 0.928$           | $-3.982 \pm 0.852$ |
| <b>BA <math>G(G_0, n, m = 2)</math></b>         | $-3.080 \pm 0.824$ | $-3.742 \pm 0.697$           | $-5.310 \pm 0.419$ |
| <b>BA <math>G(G_0, n, m = 10)</math></b>        | $-1.661 \pm 0.516$ | $-3.685 \pm 0.726$           | $-3.185 \pm 1.325$ |
| <b>WS <math>G(N, K = 2, \beta = 0.1)</math></b> | $-2.525 \pm 0.760$ | $-3.964 \pm 0.928$           | $-3.291 \pm 1.042$ |
| <b>WS <math>G(N, K = 2, \beta = 0.5)</math></b> | $-2.402 \pm 0.696$ | $-4.124 \pm 1.063$           | $-3.213 \pm 1.100$ |
| <b>WS <math>G(N, K = 2, \beta = 0.9)</math></b> | $-2.366 \pm 0.677$ | $-3.990 \pm 0.934$           | $-3.408 \pm 1.028$ |

Table 2: How dissipation rules and graph topology influence the power-law fit in avalanche size distribution. All graphs contain exactly 1000 vertices and number of grains dropped is higher than 10 millions. Values are  $k$  from  $y = ax^k$  and the error is root mean square.

From the table, the most interesting discovery is that Watts-Strogatz graphs are the most stable. Changing parameters of this creation method does not influence the power-law fit for individual dissipation rules.

The *All Once* dissipation rule does produce the lowest  $|k|$  on average, on different graph topologies. This means, the slope of the exponential function approach zero slower. On log-log plot, the line descends slower. It does so because there is a higher count of smaller avalanches. This may be explained by the average vertex degree. This rule only adds one edge to all vertices, while other rules add more on average.

Surprisingly enough, WS graphs exhibit power-law in avalanche size distributions for longest interval from all observed graphs. Based on done observations, for every combination of graph topology and



dissipation rule, the fit only holds on a sub-range. However, for WS graphs, the disturbance happens only near the max avalanche size.

## 5. Conclusion

The abelian sandpile model generalized from square lattice to an arbitrary graph does not guarantee the same power-law distribution of avalanche sizes. To ensure dissipation of sand grains from the graph, the single global sink is used with different dissipation rules. Dissipation rules dictate the number of edges from sink to all individual vertices.

From the examined combinations of graph topologies and dissipation rules could be established a few properties. It was found that for a lower relative number of edges to vertices, the avalanche size distribution fit better to a power-law.

From all discussed small-world networks, the Erdős-Renyi graph does not fulfill an important requirement, it is not a connected graph. Based on this and the nature of discussed dissipation rules, it is almost unviable for the sandpile model. However, for small edge probability ( $p = 0.1$ ) it could be used. This, however, means that ER model must be fine-tuned and thus the self-organized criticality is lost.

Both Barabási-Albert and Watts-Strogatz model generate a connected graph. Their avalanche size distribution fit nicely with a power-law and therefore this property is not limited to square lattice. It could be argued, that for these graphs it holds even better than in the original square lattice.

Dissipation rules does influence the stability of the sandpile model on the discussed graphs. One interesting is the *Fill Up* rule. This rule connects all vertices to the sink as many times necessary for the degree of all vertices to be the same. As discussed, this is advantageous for the sanpile model stability.

## 6. Technical Documentation

The accompanying interactive tool for this thesis allows users to simulate SOC on the sandpile model with all discussed graph topologies and dissipation rules. It serves primarily as a visualization of the problem with simple statistics alongside it.

The technologies used were the programming language **C++23**, with libraries *igraph*, *Dear ImGui*, *SDL3* and *OpenGL*. For a compilation was used *CMake* with *Clang* or *GCC*. Libraries are managed via *vcpkg* or as *Git* submodules. *Backward-cpp* was used for beautiful stack trace. The whole project is hosted on *GitHub*.

Naming convention:

**Classes, structs, enums** Pascal Snake Case

*Example:* Square\_Lattice

**Variables, parameters, functions, methods, namespaces** snake\_case

*Example:* sink\_rule

**Private members (or to avoid name clashes)** Trailing underscore

*Example:* avalanche\_sizes\_

The application is a visual tool and as such has an infinite render loop. The main logic is inside *App* class. This class runs the main loop and using various functions in *ui* namespace renders the right parts of the UI. Note that *Dear ImGui* is an immediate mode GUI thus the state of the UI is directly obtained from application state every time the UI is rendered.

As the application allows the user to select any combination of graph topology and dissipation rule, the state is stored in the structure *Sim\_Config*. The selected visualization method is held by *Vis\_Config*.



For the graph representation serves the *Graph* class, which stores all vertices and edges in offset and neighbours list representation, also called compressed adjacency list. That representation is fitting as no graph manipulation happens during the simulation. The Graph also holds the sand height for simulation and vertex positions for visualization.

For constructing of the graph was used the *igraph* library which already implements all discussed graph topologies, however the sink is added manually based on the specific dissipation rule.

The implementation aims to be library-independent, therefore if *igraph* were to be replaced, only the generating part has to be rewritten. Similarly, when extending the app with another graph generating library, only look inside `graph/Generator.hpp` and `graph/Converter.hpp`.

The statistics are collected using event listeners in the class *Stats\_Collector*. There are only three events, however that could be extended if desired.

The window that the application is viewed in is created via *SDL* and *OpenGL*, the minimal implementation is in `ssoc::ui::Window_Context`.

The stabilization process after a grain is dropped is handled by a queue of vertices to check. When a grain is dropped, the corresponding vertex is added to the queue. While queue is not empty, check if first vertex should topple (start an avalanche), if so do it and add all adjacent vertices to queue.

Parallelization of the stabilization process is not possible in its current form. It may be convertible to parallelizable problem. However, if interested in behaviour on one type of graph over multiple runs, it might be beneficial to run multiple simulations in parallel.

When taking measurements and running simulations, the time needed was in seconds to minutes on Intel64 Family 6 Model 191 Stepping 2 GenuineIntel 2500 Mhz Processor.

## Bibliography

- [1] R. Durrett, *Probability: Theory and Examples*, 5th ed. in Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press, 2019.
- [2] A. Clauset, C. R. Shalizi, and M. E. J. Newman, “Power-Law Distributions in Empirical Data,” *SIAM Review*, vol. 51, no. 4, pp. 661–703, 2009, doi: 10.1137/070710111.
- [3] B. D. Malamud, G. Morein, and D. L. Turcotte, “Forest Fires: An Example of Self-Organized Critical Behavior,” *Science*, vol. 281, no. 5384, pp. 1840–1842, 1998, doi: 10.1126/science.281.5384.1840.
- [4] P. Bak, *How Nature Works: The Science of Self-Organized Criticality*, 1st ed. New York, NY: Copernicus, 1996, p. XIII, 212. doi: 10.1007/978-1-4757-5426-1.
- [5] P. Bak, C. Tang, and K. Wiesenfeld, “Self-organized criticality: An explanation of the 1/f noise,” *Physical Review Letters*, vol. 59, no. 4, pp. 381–384, July 1987, doi: 10.1103/PhysRevLett.59.381.
- [6] D. J. Watts and S. H. Strogatz, “Collective dynamics of “small-world” networks,” *Nature*, vol. 393, no. 6684, pp. 440–442, 1998, doi: 10.1038/30918.
- [7] Q. K. Telesford, K. E. Joyce, S. Hayasaka, J. H. Burdette, and P. J. Laurienti, “The ubiquity of small-world networks.” [Online]. Available: <https://arxiv.org/abs/1109.5454>
- [8] A. E. Holroyd, L. Levine, K. Mészáros, Y. Peres, J. Propp, and D. B. Wilson, “Chip-Firing and Rotor-Routing on Directed Graphs,” in *In and Out of Equilibrium 2*, Birkhäuser Basel, 2008, pp. 331–364. doi: 10.1007/978-3-7643-8786-0\_17.
- [9] D. Dhar, “Self-Organized Critical State of Sandpile Automaton Models,” *Physical review letters*, vol. 64, pp. 1613–1616, 1990, doi: 10.1103/PhysRevLett.64.1613.

- [10] W.-K. Chen, *Graph Theory and Its Engineering Applications*. Singapore; River Edge, NJ: World Scientific, 1997.
- [11] P. L. Erdős and A. Rényi, “On random graphs. I,” *Publicationes Mathematicae Debrecen*, 2022, [Online]. Available: <https://api.semanticscholar.org/CorpusID:253789267>
- [12] A.-L. Barabási and R. Albert, “Emergence of Scaling in Random Networks,” *Science*, vol. 286, no. 5439, pp. 509–512, Oct. 1999, doi: 10.1126/science.286.5439.509.
- [13] E. Ivashkevich and V. Priezzhev, “Introduction to the Sandpile Model,” *Physica A: Statistical Mechanics and its Applications*, vol. 254, pp. 97–116, 1998, doi: 10.1016/S0378-4371(98)00012-0.